

# Building makemore Part 5： Building a WaveNet

从零构建 makemore 第五部分：构建 WaveNet（字幕版）｜用膨胀因果卷积打造字符级语言模型

时长 56:21 | 句数 598 | 术语 31 | 章节 16


**摘要**

本讲共 16 章，从「隐藏层 · 实现」到「隐藏层 · 神经网络」。全文 598 句、598 句已译成中文，标注了 32 个 AI 术语（主要集中在神经网络基础、数学与微积分、训练与优化）。

16 chapters, 598 sentences (598 translated), 32 AI terms annotated. Built fully offline with opus-mt + local glossary.


## 隐藏层 · 实现 00:00

00:00 Hi everyone. Today we are continuing our implementation of Make More, our favorite character-level
大家好,今天我们继续 执行《做更多》

> **implementation**
> 实现 实现
> 代码与工程

00:05 language model. Now you'll notice that the background behind me is different, that's because I am in
语言模式。 现在你会注意到我身后的背景不同, 因为我在

00:09 Kyoto and it is awesome. So I'm in the hotel room here. Now over the last few lectures we've built up
京都,它非常棒。所以我在这里的酒店房间里。现在,在最后几场演讲中,我们建立了

00:15 to this architecture that is a multi- layer perceptron character-level language model. So we see that
是一个多层的 立方字字符级语言模式。 所以我们可以看到

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
> 神经网络基础

00:21 it receives three previous characters and tries to predict the fourth character in a sequence
它接收了三个前三个字符, 并试图用序列来预测第四个字符

00:25 using a very simple multi- layer perceptron using one hidden layer of neurons with tenational
使用一个带有电信的隐蔽神经元层,使用非常简单的多层多层感官

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

**hidden layer**
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

00:30 nearnesses. So what we'd like to do now in this lecture is I'd like to complexify this architecture.
近距离。所以我们现在想在这个演讲中做的是 我想把这个建筑变得复杂

00:35 In particular we would like to take more characters in a sequence as an input, not just three,
特别是,我们想在顺序上把更多的人物作为一个输入,而不是仅仅三个

## 数据集 · 隐藏层 00:40

00:40 and in addition to that we don't just want to feed them all into a single hidden layer
此外,我们不想仅仅把他们 全部都喂进一个隐藏的层

**hidden layer**
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

00:44 because that squashes too much information too quickly. Instead we would like to make a deeper
因为那样的信息信息太过过多, 信息速度太快。 相反,我们想更深入地了解

00:49 model that progressively fuses this information to make its guess about the next character in
将信息逐渐结合到此信息的模型, 以猜测下一个字符

00:54 a sequence. And so we'll see that as we make this architecture more complex we're actually going to
序列。所以我们可以看到, 当我们让这个建筑更复杂的时候, 我们实际上会去

01:00 arrive at something that looks very much like a wave net. So wave net is this paper published by
形成一个非常像波网的东西。

01:05 Define in 2016 and it is also a language model basically but it tries to predict audio sequences
2016年定义, 并且基本上也是一种语言模式, 但试图预测音频序列

01:12 instead of character-level sequences or word-level sequences. But fundamentally the
而不是字符级序列或字级序列。但从根本上来说,

01:18 modeling setup is identical. It is an autoregressive model and it tries to predict the
模型设置相同。 它是一个自动递退模式, 试图预测

01:23 next character in a sequence. And the architecture actually takes this interesting
下一个字符在顺序中按顺序排列。而建筑结构实际上需要这个有趣的

01:27 hierarchical approach to predicting the next character in a sequence with this tree-like structure.
使用树形结构来预测下一个字符的顺序。

01:34 And this is the architecture and we're going to implement it in the course of this video.
这就是建筑,我们将在视频中实施。

01:39 So let's get started. So the starter code for part five is very similar to where we ended up
所以让我们开始吧。所以第五部分的启动代码 和我们最后的代码非常相似

01:44 in part three. Recall that part four was the manual documentation exercise that is kind of an
回顾第四部分是人工文件工作,这是一种

**recall** [ri'kɒ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

01:49 aside. So we are coming back to part three, copy pasting chunks out of it and that is our
。所以我们又回到第三部分,从第三部分中复制上铺块,这就是我们的

01:54 starter code for part five. I've changed very few things otherwise. So a lot of this should
第五编的启动代码。我改变过很少其他东西。所以很多这些应该

01:58 look familiar to you if you've gone through part three. So in particular, very briefly,
如果你读完第三部分,你就会觉得熟悉。

02:02 we are doing imports. We are reading our data set of words and we are processing the data set
我们正在做进口。我们正在读我们的一组数据单词,我们正在处理数据集

**data set**
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

02:08 of words into individual examples and none of this data generation code has changed.
单词化为单个例子,而这些数据生成代码没有变化。

02:13 And basically we have lots and lots of examples. In particular, we have 182,000 examples of three
基本上我们有很多很多的例子。特别是,我们有182,000个例子

02:20 characters trying to predict the fourth one. And we've broken up every one of these words into
试图预测第四个字的字符

02:26 little problems of given three characters predict the fourth one. So this is our data set and this
给三个字符的小问题小, 有三个字符预示第四个字符。 因此这是我们的数据集和这个

**data set**
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

02:30 is what we're trying to get the neural lab to do. Now in part three, we started to develop our
也就是我们试图让神经实验室做的工作。

02:35 code around these layer modules that are for example a class linear. And we're doing this
围绕这些层模块的代码, 这些模块是类线性。 我们正在这样做

**layer** ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

02:42 because we want to think of these modules as building blocks and like a Lego building block bricks
因为我们想把这些模块 看作是建筑砖块 就像乐高建筑砖块

02:48 that we can sort of like stack up into neural networks. And we can feed data between these
我们可以把数据输入到这些

02:52 layers and stack them up into sort of graphs. Now we also developed these layers to have
并将其堆叠成图表。现在,我们开发了这些图层

02:59 APIs and signatures very similar to those that are found in PyTorch. So we have torched.nn
API和签名与皮托尔奇的非常相似。 所以我们烧了。nn

**pytorch**
PyTorch PyTorch
代码与工程

03:04 and it's got all these layer building blocks that you would use in practice.
它有这些层块, 供你实践使用。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

03:08 And we were developing all of these to mimic the APIs of these. So for example, we have linear.
我们开发了所有这些来模仿这些API。例如,我们有线性信息。

03:13 So there will also be a torched.nn.linear and its signature will be very similar to our signature
也将会有一条被点燃的.nn.linear 和它的签字将非常相似

03:19 and the functionality will be also quite identical as far as I'm aware. So we have the linear layer
并且功能也完全相同,据我所知。所以我们有线性层

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

03:24 with the vasterm1d layer and the 10h layer that we developed previously.
与我们以前开发的10H层 和巨大的1D层

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

## 范数 · 神经网络 03:29

03:29 And linear just does a matrix multiply in the forward pass of this module.
线性只是在这个模块的前端传递中增加一个矩阵。

**matrix**
矩阵 矩阵
数学与微积分

**forward pass**
前向传播 前向传播
神经网络基础

03:33 A batch number of course is this crazy layer that we developed in the previous lecture.
当然,我们上一期讲座所开发的这个疯狂层是分批数。

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**layer** ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

03:38 And what's crazy about it is, well, there's many things. Number one, it has these running
疯狂的是,有很多事情。第一,它有这些运行

03:43 mean and variances that are trained outside of back propagation. They are trained using
在背后传播之外培训的中值差异和中值差异。

03:49 exponential moving average inside this layer when we call the forward pass .
当我们拨打前方过道时, 指数平均移动水平在此层中 。

**layer** ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

**forward pass**
前向传播 前向传播
神经网络基础

03:54 In addition to that, there's this training flag because the behavior of
除此之外,还有这个训练旗帜 因为

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

03:58 batch from is different during train time and evaluation time. And so suddenly we have
火车时间和评估时间的分批量不同。所以突然间我们有了

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

04:02 to be very careful that batch norm is in its correct state, that it's in the evaluation state
一定要非常小心,批量规范的状态是正确的, 在评估状态

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**norm**
范数 范数
数学与微积分

04:06 or training state. So that's something to now keep track of, something that sometimes introduces bugs
或训练状态。所以现在可以追踪到, 有时候会引入错误

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

04:11 because you forget to put it into the right mode. And finally we saw that batch norm couples the
最终我们看到了那些普通的批量夫妇

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**norm**
范数 范数
数学与微积分

04:16 statistics or the activations across the examples in the batch. So normally we thought of
批批中各示例中的统计或激活。所以通常我们想到

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

04:22 the batch as just an efficiency thing. But now we are coupling the computation across
批量只是效率问题,但现在我们把计算结果合并到一起

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

04:28 batch elements. And it's done for the purposes of controlling the activation statistics as we saw
批量元素。它是为了控制 我们所看到的激活数据

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

04:33 in the previous video. So it's a very weird layer, at least a lot of bugs. Partly for example,
上一个视频中。所以这是一个非常奇怪的层, 至少有很多虫子。例如,

**layer** ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

04:39 because you have to modulate the training in eval phase and so on. In addition, for example,
需要调整培训的变现阶段等。此外,例如

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

04:45 you have to wait for the mean and the variance to settle and to actually reach a steady state.
您必须等待平均值和差异的解决,并真正达到稳定状态。

04:51 And so you have to make sure that you basically there's state in this layer and state is harmful.
所以你必须保证你基本上 处于这种状态, 并且状态是有害的。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

04:56 Now, I brought out the generator object. Previously we had a generator equals G and so on inside these
之前我们有发电机,所以在它们里面

05:05 layers. I've discarded that in favor of just initializing the torch RNG outside here just once
我放弃了这个, 同意在这里刚刚开始 红核的火炬

05:13 globally, just for simplicity. And then here we are starting to build out some of the neural
在全球范围,只是为了简单。然后我们开始建立一些神经

05:18 network elements. This should look very familiar. We are, we have our embedding table C. And then
网络元素的网络元素。 这应该看起来很熟悉。 我们, 我们有嵌入表C。 然后

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

05:24 we have a list of players. And it's a linear feeds to bachelor feeds to 10 H. And then a linear output
我们有一张玩家列表。它是一个线性种子 向单身汉输入到10H。然后是线性输出

05:31 layer. And it's weights are scaled down. So we are not confidently wrong at initialization.
。 并且它被缩放的重量。 所以我们在初始化时没有自信错误。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

05:36 We see that this is about 12,000 parameters. We're telling pytorch that the parameters require
我们看到这大约是12,000个参数。我们告诉皮火公司,这些参数需要

**pytorch**
PyTorch PyTorch
代码与工程

05:41 gradients. The optimization is as far as I'm aware, identical and should look very, very familiar.
梯度。 据我所知,优化是相同的,应该非常熟悉

05:47 Nothing changed here. Last function looks very crazy. We should probably fix this.
这里没有变化 最后一项功能看起来非常疯狂 我们也许应该解决这个问题

05:53 And that's because 32 batch elements are too few. And so you can get very lucky or unlucky in any one
这是因为32批元素太少。所以你就可以在任何一个中 变得非常幸运或不幸

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

06:00 of these batches. And it creates a very thick loss function. So we're going to fix that soon.
产生非常厚的损失功能。所以我们很快会修好它

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:06 Now, once we want to evaluate the train neural network, we need to remember, because of the
现在,一旦我们想评估火车神经网络 就需要记住

**neural network**
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

06:10 bachelor layers, to set all the layers to be training equals false. This only matters for
学士级,设置所有层次的培训为培训等假的。

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

06:15 the bachelor layer so far. And then we evaluate. We see that currently we have validation loss
至今为止的单身汉层。然后我们进行评估。我们看到,目前我们有验证损失

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

**validation**
验证 验证
训练与优化

06:22 of 2.10, which is fairly good, but there's still a ways to go. But even at 2.10, we see that when
0.10,这是相当不错的,但是还有一条路要走。但即使到了2.10,我们可以看到

06:29 we sample from the model, we actually get relatively name like results that do not exist in a training
我们从模型中抽样,我们实际上得到相对的名称 就像训练中不存在的结果

**sample**
样本 样本
神经网络基础

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

06:34 set. So for example, if on kilo, a pros, a liar, et cetera. So certainly not reasonable,
设此例,如果以公斤计,是专业的,是说谎的。这确是不合理的。

06:45 not unreasonable, I would say, but not amazing. And we can still push this validation loss even
并不是不合理,但并不令人惊叹。我们还可以推这个论证损失甚至

**validation**
验证 验证
训练与优化

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

06:49 lower and get much better samples that are even more name like. So let's improve this model.
并且得到更好的样本, 更像名称。让我们改进这个模型。

## 学习速率 · 召回率 06:56

06:56 Okay, first let's fix this graph because it is daggers in my eyes and I just can't take it anymore.
首先,让我们修正这个图表 因为它是我眼中的匕首 而我再也受不了了

07:01 So last, if you recall, is a Python list of floats. So for example, the first 10 elements
所以,如果你记得,最后的,是 Python 浮标列表。例如,前十个元素

**recall** [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

07:09 look like this. Now, what we'd like to do basically is we need to average up
现在,我们想做的是平均

07:14 some of these values to get a more sort of representative
某些价值观可以让自己更具代表性

07:18 value along the way. So one way to do this is the following. In PyTorch, if I create, for example,
沿路值。所以做这个的一个方法就是如下。在 PyTorrch, 如果我创建,例如,

**pytorch**
PyTorch PyTorch
代码与工程

07:24 a tensor of the first 10 numbers, then this is currently a one dimensional array. But recall
a 前十个数字的 10 的 10, 那么这是目前一个维数组。 但是记住

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**recall** [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

07:30 that I can view this array as two dimensional. So for example, I can use it as a two by five array.
,我可以将这个阵列作为两个维。例如,我可以用它作为两个乘五个阵列。

07:35 And this is a 2D tensor now, two by five. And you see what PyTorch has done is that the first row
这是二维加长,两乘五。你看,皮托尔奇做的就是第一排

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**pytorch**
PyTorch PyTorch
代码与工程

07:41 of this tensor is the first five elements. And the second row is the second five elements.
中的第一个是前五个元素。第二行是第二个元素。

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

07:46 I can also view it as a five by two as an example. And then recall that I can also
我也可以把它看作是一个五乘二的例子。然后我记得,我可以也

**recall** [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

07:52 use negative one in place of one of these numbers. In PyTorch, we'll calculate what that number must
使用负号一个代替其中的数字之一。在 PyTorch, 我们将计算该数字必须多少

**pytorch**
PyTorch PyTorch
代码与工程

07:58 be in order to make the number of elements work out. So this can be this or like that, both
使元素的数量能够解决。所以,这可以是这样或那样,两者都是

08:05 will work. Of course, this would not work. Okay, so this allows it to spread out some
当然,这行不通。好吧,这样可以让它传播一些

08:11 of the consecutive values into rows. So that's very helpful because what we can do now is,
将连续的值排成行。所以这非常有用,因为我们现在可以做的是,

08:16 first of all, we're going to create a torch shot tensor out of the list of floats. And then we're
首先,我们要从浮标中 产生一个火炬,发射气压

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

08:22 going to view it as whatever it is, but we're going to stretch it out into rows of 1000 consecutive
把它看成是任何事物 但我们要把它排成1000列

08:29 elements. So the shape of this now becomes 200 by 1000. And each row is 1000 consecutive
元素。因此,此形状的形状现在变成200乘1000。每行是1000连续

08:38 elements in this list. So that's very helpful because now we can do a mean along the rows.
列表中的元素。所以这非常有用, 因为现在我们可以在行上做一个刻薄的动作了。

08:44 And the shape of this will just be 200. And so we've taken basically the mean on every row.
其形状将只有200。所以我们基本上采取了每行的平均值。

08:49 So PLT dot plot of that should be something nicer. Much better. So we see that we've basically
那样PLT点的图案应该更好一些。好多了。所以我们看到我们基本上已经

08:56 made a lot of progress. And then here, this is the learning rate decay. So here we see that
取得了很大的进步。然后,这里,这就是学习率的衰减。我们在这里可以看到

**learning rate**
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

09:01 the learning rate decay subtracted a ton of energy out of the system and allowed us to settle
学习率衰减 减去了系统一吨能量 并让我们得以和解

**learning rate**
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

09:06 into sort of the local minimum and this optimization. So this is a much nicer plot.
排序为本地最小值和优化。 因此这是一个更漂亮的图案 。

09:11 Let me come up and delete the monster. And we're going to be using this going forward.
让我上去删除那个怪物,我们用这个前进

09:16 Now next up, what I'm bothered by is that you see our forward pass is a little bit gnarly
接下来,我担心的是 你看到我们的前方通道 有点刺痛

**forward pass**
前向传播 前向传播
神经网络基础

09:22 and takes way too many lines of code. So in particular, we see that we've organized some
并且采取了太多的代码线。所以,我们特别看到,我们已经组织了一些

## 运算 · PyTorch 09:26

09:26 of the layers inside the layers list, but not all of them for no reason. So in particular,
而不是无缘无故地全部。所以,

09:32 we see that we still have the embedding table special case outside of the layers. And in addition
我们看到,我们还有 嵌入表的特殊情况 在层外。此外

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

09:37 to that, the viewing operation here is also outside of our layers. So let's create layers for these.
至此,这里的浏览操作也不属于我们的层次。所以让我们为这些层次建立层。

**operation**
运算 运算
神经网络基础

09:43 And then we can add those layers to just our list. So in particular, the two things that we need is
然后我们可以把这些层次加到我们的清单中。所以,特别是,我们需要的两件事是

09:48 here, we have this embedding table. And we are indexing at the integers inside the batch XB
这儿,我们有一个嵌入表。我们正在对批量内整数进行索引

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

09:56 inside the tensor XB. So that's an embedding table lookup just done with indexing. And then here we
里面的 ARO 预算外。 所以这是一个嵌入式表格的搜索, 刚刚用索引完成。 然后我们在这里

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

10:03 see that we have this view operation, which if you recall from the previous video, simply rearranges
我们看到我们有一个查看操作, 如果你记得上次视频里的话, 只需重新排列

**operation**
运算 运算
神经网络基础

**recall** [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

10:08 the character embeddings and stretches them out into row. And effectively, what that does is the
字符嵌入并把它们拉伸成行。实际上,它的作用是

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

10:15 concatenation operation, basically, except it's free because viewing is very cheap in PyTorch.
基本上,除了免费外, 康纳化手术是免费的 因为皮托尔奇的观光很便宜

**operation**
运算 运算
神经网络基础

**pytorch**
PyTorch PyTorch
代码与工程

10:21 And no memories being copied. We're just re-representing how we view that tensor.
也没有复制记忆 我们只是重新陈述我们如何看待这些压力

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

10:25 So let's create modules for both of these operations, the embedding operation and the
因此让我们为这两个操作, 嵌入操作和

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

**operation**
运算 运算
神经网络基础

10:32 flattening operation. So I actually wrote the code in just to save some time. So we have a module
平坦操作。 所以我实际上写了代码,只是为了节省一些时间。 所以我们有一个模块

**operation**
运算 运算
神经网络基础

10:39 embedding and a module flatten. And both of them simply do the indexing operation in the
嵌入和一个模块平板。他们两个都只是做

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

**operation**
运算 运算
神经网络基础

10:44 forward pass and the flattening operation here. And this C now will just become a salt dot weight
前面的过道和这里的平坦操作。 而这个C现在将变成盐点重量

**forward pass**
前向传播 前向传播
神经网络基础

**operation**
运算 运算
神经网络基础

10:54 inside an embedding module. And I'm calling these layers specifically embedding and flattened
嵌入模块。 我将这些层称为 专门嵌入和平坦

**embedding** [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

10:59 because it turns out that both of them actually exist in PyTorch. So in PyTorch, we have nn.embedding.
因为他们两个都存在于皮托尔奇。所以在皮托尔齐,我们有NN的编队。

**pytorch**
PyTorch PyTorch
代码与工程

**embedding** [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

11:05 And it also takes the number of embeddings and the dimensionality of the embedding,
也包含嵌入的数量和嵌入的维度

**embedding** [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

11:09 just like we have here. But in addition, PyTorch takes a lot of other keyword arguments
皮托尔奇也用了很多其他关键词来论证

**pytorch**
PyTorch PyTorch
代码与工程

11:13 that we are not using for our purposes yet. And for flatten, that also exists in PyTorch.
并且为了平坦,在皮托尔奇也存在。

**pytorch**
PyTorch PyTorch
代码与工程

11:20 And it also takes additional keyword arguments that we are not using. So we have a very simple
还需要我们没有使用的额外的关键词参数。所以我们有一个非常简单的

11:24 flatten. But both of them exist in PyTorch. They're just a bit more simpler. And now that we have
平托尔奇有两张,更简单一点

**pytorch**
PyTorch PyTorch
代码与工程

11:31 these, we can simply take out some of these special case things. So instead of C, we're just
这些,我们可以简单地拿出一些这些特殊情况。所以我们不是C,而是C

11:39 going to have an embedding and a vocab size and n.embed. And then after the embedding, we are going
将有一个嵌入器,一个孔卡大小和编织。然后在嵌入后,我们要去

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

11:46 to flatten. So let's construct those modules. And now I can take out this C. And here I don't have
来平坦。所以让我们建造这些模块。现在我可以拿出这个C。而在这里,我没有

11:54 to special case it anymore, because now C is the embeddings weight. And it's inside layers.
特殊情况,现在C是嵌入重量。它属于内部层。

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

**weight** [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

12:01 So this should just work. And then here, our forward pass simplifies substantially,
所以这应该能奏效。然后,我们向前的通道大大简化了

**forward pass**
前向传播 前向传播
神经网络基础

12:07 because we don't need to do these now outside of these layer outside and explicitly,
因为我们不需要现在做这些 现在外面的这个层外, 和明确的,

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

12:13 they're now inside layers. So we can delete those. But now to kick things off, we want this little X,
现在它们都进入了内部层。所以我们可以删除这些。但是现在,为了把东西踢开,我们要这个小X,

12:20 which in the beginning is just XP, the tensor of integers specifying the identities of these
开始时只为 XP, 表示这些

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

12:25 characters at the input. And so these characters cannot directly feed into the first layer.
输入字符。所以这些字符无法直接输入第一个层。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

12:30 And this should just work. So let me come here and insert a break, because I just want
这应该能奏效。让我来这里 插入一个休息,因为我只是想

12:35 to make sure that the first iteration of this runs and that there's no mistake. So that ran properly.
以确保第一次重复运行,确保没有错误。所以正常运行。

12:41 And basically, we've substantially simplified the forward pass here. Okay, I'm sorry, I changed my
基本上,我们已经大大简化了前方通行证

**forward pass**
前向传播 前向传播
神经网络基础

12:46 microphone. So hopefully the audio is a little bit better. Now, one more thing that I would like to
麦克风。希望音响会好一点,现在,我还想再做一件事

## 对数 · 样本 12:51

12:51 do in order to pytorchify our code even further is that right now we are maintaining all of our
这样做是为了把我们的法典火化,甚至进一步。

12:55 modules in a naked list of layers. And we can also simplify this, because we can introduce the
模块,在裸露的层列列表中。我们也可以简化这个程序,因为我们可以引入

13:01 concept of pytorch containers. So in torch.nn, which we are basically rebuilding from scratch here,
燃烧炉容器的概念。所以在火炬.nn, 我们基本上正在从这里的零开始重建,

**pytorch**
PyTorch PyTorch
代码与工程

13:07 there's a concept of containers. And these containers are basically a way of organizing
有一种集装箱的概念,这些集装箱基本上是一种组织方式

13:11 layers into lists or dicts and so on. So in particular, there's a sequential,
以列表或文字等形式, 以图图示。 所以, 特别是, 有顺序,

13:17 which maintains a list of layers. And it is a module class in pytorch. And it basically
保存一个层次列表。 它是一个模块类, 在 Pytorch 中。 它基本上

**pytorch**
PyTorch PyTorch
代码与工程

13:23 just passes a given input through all the layers sequentially, exactly as we are doing here.
只需通过一个输入 通过所有层次的相继传输, 和我们在这里所做的一样。

13:28 So let's write our own sequential. I've written a code here. And basically, the code for sequential
所以让我们写出我们自己的相继顺序。 我在这里写了一个代码。 基本上, 顺序的代码

13:34 is quite straightforward. We pass in a list of layers, which we keep here. And then given any
很简单,我们通过一系列的层次, 我们保留在这里。然后给任何

13:39 input in a forward pass, we just call all the layers sequentially and return the result.
输入前传, 我们按顺序调用所有层, 然后返回结果 。

**forward pass**
前向传播 前向传播
神经网络基础

13:44 In terms of the parameters, it's just all the parameters of the child modules.
以参数来说,它只是儿童模块的所有参数。

13:48 So we can run this. And we can again, simplify this substantially, because we don't
所以我们可以运行这个。我们可以再次, 简化它,因为我们不

13:52 maintain this naked list of layers, we now have a notion of a model, which is a module.
保持这个裸体的层列, 我们现在有了模型的概念, 这是一个模块。

13:58 And in particular is a sequential of all the layers. And now parameters are simply just a model
特别是所有层次的顺序。现在参数只是个模型

14:07 that parameters. And so that list comprehension now lives here. And then here we are press
参数。所以列表理解 现在就住在这里,然后我们按

14:15 here we are doing all the things we used to do. Now here, the code again, simplifies substantially,
我们在这里做我们过去所做的一切事情。现在,这里的代码再次大大简化,

14:20 because we don't have to do this forwarding here, instead we just call the model on the input data.
因为我们不必在这里进行这个传输, 而是我们只用输入数据模型来调用。

14:26 And the input data here are the integers inside xb. So we can simply do logits, which are the outputs
这里输入的数据是 xb 中的整数。 所以我们可以简单地做登录, 这些就是输出

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

14:32 of our model, are simply the model called on xb. And then the cross entropy here takes the
也就是我们模型的模型, 简单的模型被调用 xb。 然后这里的交叉星

14:38 logits and the targets. So this simplifies substantially. And then this looks good. So
日志和目标。 所以这个简单化了。 然后看起来不错。 所以这样。

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

14:45 let's just make sure this runs. That looks good. Now here, we actually have some work to do still here,
我们先确保这支队伍的运行,这样看起来不错。现在,我们还有工作要做,

14:51 but I'm going to come back later. For now, there's no more layers, there's a model that layers,
现在没有一层层了,有模型可以分层

14:56 but it's not a to access attributes of these classes directly. So we'll come back and fix
但它不是直接访问这些课程属性的选项。 所以我们会回来,解决

15:01 this later. And then here, of course, this simplifies substantially as well, because logits are
稍后这个。当然,这里,这里,这个也大大简化了, 因为登录是

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

15:07 the model called on x. And then these logits come here. So we can evaluate the train
模型调用 x。 然后这些登录器就来了这里。 所以我们可以评估火车

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

15:15 and validation loss, which currently is terrible because we just initialize the neural net.
和验证损失, 目前这是可怕的,因为我们刚刚开始 神经网。

**validation**
验证 验证
训练与优化

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

15:20 And then we can also sample from the model. And this simplifies dramatically as well,
然后我们也可以从模型中取样。这个简化也非常简单,

**sample**
样本 样本
神经网络基础

15:24 because we just want to call the model onto the context and outcome logits. And then these
因为我们只是想把模型 调上背景和结果记录。然后这些

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

15:30 logits go into softmax and get the probabilities, etc. So we can sample from this model.
登录进入软负乘,并获得概率等。 所以我们可以从这个模型中取样。

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

**sample**
样本 样本
神经网络基础

15:38 What did I screw up? Okay, so I fixed the issue and we now get the result that we expect,
我搞砸了什么?

15:45 which is gibberish, because the model is not trained because we re initialize it from scratch.
胡言乱语,因为模型没有经过训练 因为我们从头开始重新初始化

15:50 The problem was that when I fixed this cell to be modeled out layers instead of just layers,
问题是当我把这个细胞固定在 建模层上时, 而不是仅仅建层

## 训练 · 范数 15:55

15:55 I did not actually run the cell. And so our neural net was in a training mode. And what caused
手机实际上不是我运行的。所以我们的神经网处于训练模式中。

**neural net**
神经网络 神经网络
神经网络基础

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

16:00 the issue here is the bathroom layer as bathroom layer often likes to do, because
问题是浴室层 因为浴室层经常喜欢这样做,因为

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

16:05 bathroom was in a training mode. And here we are passing in an input, which is a batch of
浴室是训练模式的。我们在这里通过一个输入,这是一组

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

16:10 just a single example made up of the context. And so if you are trying to pass in a single
只有一个例子,由上下文组成。所以如果你试图通过一个单一的

16:14 example into a batch norm that is in the training mode, you're going to end up estimating the variance
示例 , 以作为批量规范。 在培训模式中, 您最终会估计差异

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**norm**
范数 范数
数学与微积分

16:19 using the input. And the variance of a single number is not a number, because it is a measure
输入。而一个数字的差数不是一个数字,因为它是一个量度

16:24 of a spread. So for example, the variance of just a single number five, you can see is not a
。例如,一个位的差数, 以五点之间的差数为例, 你可以看到,它不是

16:29 number. And so that's what happened. And the bathroom basically caused an issue. And then
和浴室引发的问题。然后

16:35 that polluted all of the further processing. So all that we had to do was make sure that
污染了所有进一步处理过程。所以我们要做的就是确保

16:39 this runs. And we basically made the issue of, again, we didn't actually see the issue with the loss.
我们基本上又把损失的问题 放在了一起,我们没有看到损失问题。

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:47 We could have evaluated the loss, but we got the wrong result because the bathroom was in the
我们可以评估损失, 但我们得到错误的结果 因为浴室在

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

16:50 training mode. And so we still get a result, which is the wrong result because it's using the
训练模式。所以我们还是会得到一个结果, 这是错误的结果, 因为它使用

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

16:57 sample statistics of the batch, whereas we want to use the running mean and running variance
批量抽样统计样本, 而我们想要使用运行中值和变化

**sample**
样本 样本
神经网络基础

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

17:01 inside the bathroom. And so again, an example of introducing a bug inline, because we did
浴室内,所以再次,一个引入错误插入的示例, 因为我们做了

17:08 not properly maintain the state of what is training or not. Okay, so I rerun everything.
训练与否的状况没有适当维持

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

17:13 And here's where we are. As a reminder, we have the training loss of 2.05 and validation 2.10.
提醒一下,我们损失了2.05和2.10

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

17:18 Now, because these losses are very similar to each other, we have a sense that we are not
现在,由于这些损失彼此非常相似,我们感到我们不是

17:22 overfitting too much on this task. And we can make additional progress in our performance
这项任务过于过重地适应了我们的任务。

## 缩放 · 神经网络 17:27

17:27 by scaling up the size of the neural network and making everything bigger and deeper.
扩大神经网络的规模 使一切变得更大更深入

**scaling** ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

**neural network**
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

17:32 Now, currently we are using this architecture here, where we are taken in some number of
现在,我们目前在这里使用这个建筑, 在那里我们被带入了

17:36 characters going into a single hidden layer and then going to the prediction of the next character.
字符进入一个隐藏的层,然后进行下一个字符的预测。

**hidden layer**
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

17:41 The problem here is we don't have a naive way of making this bigger in a productive way. We could,
问题是我们没有天真的方法 来让这个大起来, 以富有成效的方式。

17:47 of course, use our layers, sort of building blocks and materials to introduce additional layers
当然,用我们的层、 某种构件和材料来引进更多的层

17:53 here and make the network deeper, but it is still the case that we are crushing all of the
并且让网络更深的深度, 但我们仍然在摧毁所有

17:57 characters into a single layer all the way at the beginning. And even if we make this a
字符字符在开始的单层中一直贯穿于一个单独的层。即使我们把它变成

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

18:02 bigger layer and add neurons, it's still kind of like silly to squash all that information so fast
更大的层层加神经元, 仍然有点傻 将所有信息都拼得这么快

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

18:08 in a single step. So what we'd like to do instead is we'd like our network to look a lot more like
单步走一步。所以我们想做的是 我们希望我们的网络看起来更像

18:13 this in the WaveNet case. So you see in the WaveNet, when we are trying to make the prediction for
也就是在WaveNet中,我们试图预测

18:17 the next character in the sequence, it is a function of the previous characters that feed in,
序列中的下一个字符,是前几个输入的字符的函数,

18:23 but not all of these different characters are not just crushed to a single layer and then
但不是所有这些不同的字符 不只是压碎到一层层 然后

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

18:28 you have a sandwich. They are crushed slowly. So in particular, we take two characters and we fuse
特别是,我们用两个字符和引信

18:34 them into sort of like a bi-gram representation and we do that for all these characters consecutively
都变成像一个二-克代表,我们做 所有这些字符连续

18:40 and then we take the bi-grams and we fuse those into four character level chunks and then we
然后我们拿了二克的字数 然后把这些组合成四个字符级块 然后我们

18:47 fuse that again. And so we do that in this like tree-like hierarchical manner. So we fuse
重新结合,我们用树一样的等级方式来做。所以我们用木形的高度方式来做

18:52 the information from the previous context slowly into the network as it gets deeper. And so this is
上一个背景的信息随着网络的深度不断加深而慢慢地进入网络。所以这是

18:58 the kind of architecture that we want to implement. Now in the WaveNet's case, this is a visualization
一种我们想要实施的建筑。现在在WaveNet的案例中,这是一个可视化

19:03 of a stack of dilated causal convolution layers. And this makes it sound very scary, but actually
一系列膨胀的因果卷变层。这听起来很可怕,但实际上

**convolution** [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

19:08 the idea is very simple. And the fact that it's a dilated causal convolution layer is really
其想法很简单。事实上,它是一个膨胀的因果变化层

**convolution** [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

19:13 just an implementation detail to make everything fast. We're going to see that later. But for
执行细节,可以让一切快速地进行

**implementation**
实现 实现
代码与工程

19:17 now, let's just keep the basic idea of it, which is this progressive fusion. So we want to make
现在,让我们保持它的基本概念, 这就是这个进步的融合。 所以我们想让

19:22 the network deeper. And at each level, we want to fuse only two consecutive elements,
网络更深。在每一层次, 我们只想要连接两个连续的元素,

19:27 two characters, then two bi-grams, then two four-grams, and so on. So let's implement this.
两个字符,然后两个二维格, 然后两个四公克等等。让我们来执行这个。

19:32 Okay, so first up, let me scroll to where we built the dataset and let's change the block size from
首先,让我滚到我们建数据集的地方 换个区块大小

19:36 three to eight. So we're going to be taking eight characters of context to predict the ninth
3到3到8。所以我们用八个字符的上下文来预测第九

19:42 character. So the dataset now looks like this. We have a lot more context feeding in to predict
字符。因此数据集现在看起来是这样的。我们有很多更多的上下文供人使用来预测

19:47 any next character in a sequence. And these eight characters are going to be processed in this
任意下一个字符序列中的任意下一个字符。 这八个字符将在此处理

19:51 tree-like structure. Now, if we scroll here, everything here should just be able to work. So
树状结构。如果我们滚动到这里, 这里的一切应该都能工作。所以

19:57 we should be able to redefine the network. You see that the number of parameters has increased by
网络应该可以重新定义。

20:01 10,000. And that's because the block size has grown. So this first linear layer is much,
10,000. 那是因为块面积已经扩大。所以第一个线性层是很大的,

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
> 神经网络基础

20:06 much bigger. Our linear layer now takes eight characters into this middle layer. So there's
更大的。 我们的线性层现在需要八个字符进入中间层。 因此,

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
> 神经网络基础

20:12 a lot more parameters there. But this should just run. Let me just break right after the very
更多的参数。但这个应该运行。让我在

20:18 first iteration. So you see that this runs just fine. It's just that this network doesn't make too
第一次迭代。所以你可以看到,这个运行很好。只是这个网络没有做太

20:22 much sense. We're crushing way too much information, way too fast. So let's now come in and see how
we
感觉很深。我们正在粉碎的方式太多的信息,太多的信息,太快了。所以让我们来看看如何

20:28 could try to implement the hierarchical scheme. Now, before we dive into the detail of the
尝试执行等级制度。现在,在我们潜入

20:33 re-implementation here, I was just curious to actually run it and see where we are in terms
重新实施,我只是好奇 实际运行它 看看我们在哪里

**implementation**
实现 实现
代码与工程

20:38 of the baseline performance of just lazily scaling up the context length. So I'll let it run,
光是懒散地放大上下文长度的基线性能。

**scaling** ['skeiliŋ]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

20:43 we get a nice loss curve. And then evaluating the loss, we actually see quite a bit of
损失曲线曲线的曲线,然后对损失进行评估,我们实际上看到相当

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

20:47 improvement just from increasing the context length. So I started a little bit of a performance log here.
仅从增加上下文长度来改进。 所以我开始在这里做一点性能记录。

20:53 And previously where we were is we were getting a performance of 2.10 on the validation loss.
之前我们当时的情况是 我们得到了2.10的确认损失的证明

**validation**
验证 验证
训练与优化

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

20:58 And now simply scaling up the context length from 3 to 8 gives us a performance of 2.02.
现在只是把上下文长度从3增加到8, 给我们的表演是2.02.

**scaling** ['skeilin]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

21:04 So quite a bit of an improvement here. And also when you sample from the model,
这儿的改进相当大,而且当你从模型中取样时

**sample**
样本 样本
神经网络基础

21:08 you see that the names are definitely improving qualitatively as well.
名字的质量也明显提高

21:13 So we could of course spend a lot of time here tuning things and making it even bigger and scaling
所以我们当然可以花很多时间在这里 调整事情,使它更大,更大规模

**scaling** ['skeilin]
缩放 特征工程 中的一种常用做法，是指对某个特征的值区间进行调整，使之 与数据集中其他特征的值区间一致。例如，假设您希望数据集中所有浮点特征的值都位于 0 到 1 区间内，如果某个特征的值位于 0 到 500 区间内，您就可以通过将每个值除以 500 来缩 放该特征。 另请参阅标准化 。 scikit-learn 一个热门的开放源代码机器学习平
神经网络基础

21:18 up the network further, even with the simple sort of setup here. But let's continue and let's
网络更进一步,即使在这里设置简单一些的设置。

21:24 implement here our comodal and treat this as just a rough baseline performance. But there's a lot of
实施我们的共同模式, 把它当作粗略的基线性能。

21:30 optimization like left on the table in terms of some of the hyper parameters that you're
优化像桌上剩下的一样 以一些高参数表示

21:34 hopefully getting a sense of now. Okay, so let's scroll up now and come back up. And what
好,让我们滚起来再回来

21:40 I've done here is I've created a bit of a scratch space for us to just like look at the forward
我在这里所做的就是,我已经创造了一个有点刮伤的空间 我们只是喜欢看前方

## 矩阵乘法 · 向量 21:45

21:45 pass of the neural net and inspect the shape of the tensors along the way as the neural net forwards.
神经网的口,并检查 电压的形状 沿路 随着神经网向前。

**neural net**
神经网络 神经网络
神经网络基础

21:52 So here I'm just temporarily for debugging creating a batch of just say four examples.
所以我在这里只是暂时的 调试,因为创建了一组 只是说四个例子。

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

21:57 So four random integers, then I'm plucking out those rows from our training set.
四个随机整数 然后我从训练场上拔出那几排

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

22:02 And then I'm passing into the model the input xp. Now the shape of xp here because we
然后我就跳进模型输入 xp。现在的 xp 形状在这里,因为我们

22:08 have only four examples is four by eight. And this eight is now the current block size. So
四个例子只有四个,四乘八。这八个是目前块的大小。所以现在这个数字是目前的区块大小。

22:15 inspecting xp, we just see that we have four examples. Each one of them is a row of xp.
检查 xp, 我们可以看到我们有四个例子。 每个都是一列 xp 。

22:21 And we have eight characters here. And this integer tensor just contains the identities of
我们这里有八个字符。这个整数的发光器 含有

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

22:26 those characters. So the first layer of our neural net is the embedding layer.
这些字符。所以我们神经网的第一层就是嵌入层。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

**neural net**
神经网络 神经网络
神经网络基础

22:32 So passing xp, this integer tensor through the embedding layer creates an output that is
传递 xp, 通过嵌入层的整数拉拉 产生输出

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

22:37 four by eight by 10. So our embedding table has for each character a 10 dimensional vector that we
10x8x4x4x8x10x10。 所以我们的嵌入表为每个字符 都配有10维矢量,我们

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

**vector**
向量 向量
数学与微积分

22:44 are trying to learn. And so what the embedding layer does here is it blocks out the embedding vector

正在尝试学习。 所以嵌入层在这里所做的就是 它阻挡嵌入矢量

**embedding** [em'bediŋ]

嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n

神经网络基础

**layer** ['leiə]

层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T

神经网络基础

22:51 for each one of these integers and organizes it all in a four by eight by 10 tensor now.

每个整数 并组织成四乘八乘十

**tensor** ['tensə]

张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。

数学与微积分

22:58 So all of these integers are translated into 10 dimensional vectors inside this

所以所有这些整数都转换成 10 维矢量

23:02 three dimensional tensor now. Now passing that through the flatten layer, as you recall,

现在,通过平层,你记得

**tensor** ['tensə]

张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。

数学与微积分

**layer** ['leiə]

层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T

神经网络基础

23:07 what this does is it views this tensor as just a four by eight tensor. And what that effectively does

这样做的就是它把这个阵列看成 仅仅四乘八。而那有效的作用是什么?

**tensor** ['tensə]

张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。

数学与微积分

23:14 is that all these 10 dimensional embeddings for all these eight characters just end up being
所有的这10维嵌入器 全部的8个字符

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

23:18 stretched out into a long row. And that looks kind of like a concatenation operation basically.
长长的一排,看起来就像一个接合操作

**operation**
运算 运算
神经网络基础

23:24 So by viewing the tensor differently, we now have a four by 80. And inside this 80,
所以通过不同看待变异的变异, 我们现在有四比八十 而这80中

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

23:30 it's all the 10 dimensional vectors just concatenating next to each other. And the linear
10维矢量都是相互连接的。

23:36 layer of course takes 80 and creates 200 channels just via matrix multiplication. So so far so good.
平方层的自然值需要80个,然后通过矩阵乘法创建200个频道。到目前为止,这些频道还不错。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

**matrix**
矩阵 矩阵
数学与微积分

23:44 Now I'd like to show you something surprising. Let's look at the insides of the linear layer
现在我想给你们展示一些惊喜。让我们看看线性层的内侧

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

23:50 and remind ourselves how it works. The linear layer here in the forward pass takes the input x,
并提醒自己它是如何运作的。前方的线性层 将输入 x,

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

**forward pass**
前向传播 前向传播
神经网络基础

23:56 multiplies it with a weight and then optionally adds bias. And the weight here is two dimensional
以重量乘倍,然后可选增加偏向。而这里的重量是二维

**weight** [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

**bias** ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

24:00 as defined here. And the bias is one dimensional here. So effectively in terms of the shapes involved,
此处定义的偏差是一维的。从形状来看,

**bias** ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

24:07 what's happening inside this linear layer looks like this right now. And I'm using random numbers
线性层内部发生的一切现在看起来是这样的。我使用随机数字

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

24:12 here. But I'm just illustrating the shapes and what happens. Basically a four by 80 input
这里,但我只是说明形状和发生了什么。 基本上一个四乘80输入

24:18 comes into the linear layer gets multiplied by this 80 by 200 weight matrix inside. And there's
进入线性层后, 以80乘以80乘200重量矩阵。 然后有

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

**weight** [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

24:23 a plus 200 bias. And the shape of the whole thing that comes out of the linear layer is four by 200
a + 200 的偏差。 从线性层产生的整个物体的形状是 四乘二

**bias** ['baiəs]
偏差 距离原点的截距或偏移。偏差（也称为偏差项）在机器学习模型中用 b 或 w0 表示。例如， 在下面的公式中，偏差为 b： 请勿与预测偏差 混淆。
神经网络基础

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

24:28 as we see here. Now notice here by the way that this here will create a four by 200 tensor. And then
注意这里,这样可以创造四乘200度

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

24:36 plus 200, there's a broadcasting happening here, but four by 200 broadcasts with 200. So everything
加200,这里有广播,但四、二百 和200的广播

24:42 works here. So now the surprising thing that I'd like to show you that you may not expect
在这儿工作。现在,我要给你一个惊喜, 让你看到,你可能无法预料

24:47 is that this input here that is being multiplied, it doesn't actually have to be
输入,是这个输入的乘数, 它实际上不必

24:52 two dimensional. This matrix multiply operator in PyTorch is quite powerful. And in fact, you can
平托奇的矩阵乘数操作器非常强大。事实上,你可以

**matrix**
矩阵 矩阵
数学与微积分

**pytorch**
PyTorch PyTorch
代码与工程

24:57 actually pass in higher dimensional arrays or tensors and everything works fine. So for example,
实际通过在高维数组或加压器中,一切都很好。例如,

25:02 this could be four by five by 80. And the result in that case will become four by five by 200.
可能要四五八十 这样的结果就是四五二百

25:08 You can add as many dimensions as you like on the left here. And so effectively what's
您可以在左边添加您喜欢的维度。 并且有效

25:12 happening is that the matrix multiplication only works on the last dimension. And the dimensions
发生的情况是矩阵乘法只对最后一个维值起作用。

**matrix**
矩阵 矩阵
数学与微积分

**dimension**
维度 维度
数学与微积分

25:18 before it in the input tensor are left unchanged. So that is basically these these dimensions on
输入的电压在它之前保持不变。所以基本上就是这些维度

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

25:28 the left are all treated as just a batch dimension. So we can have multiple batch dimensions. And then
左侧的左侧都被当作一个批量维度。 所以我们可以有多个批量维量。 然后

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

25:34 in parallel over all those dimensions, we are doing the matrix multiplication on the last dimension.
在所有这些层面的同时,我们正在对最后一个层面进行矩阵乘法。

**matrix**
矩阵 矩阵
数学与微积分

**dimension**
维度 维度
数学与微积分

25:39 So this is quite convenient because we can use that in our network now. Because remember that
所以这很方便,因为我们现在可以 将它用于网络。因为记住

25:45 we have these eight characters coming in. And we don't want to now flatten all of it out into a
我们这八个人物来了,我们不想现在 把所有的都压扁成

25:52 large eight dimensional vector. Because we don't want to a matrix multiply 80 into a weight matrix
宽八维矢量。因为我们不想一个矩阵将80乘以一个重量矩阵

**vector**
向量 向量
数学与微积分

**matrix**
矩阵 矩阵
数学与微积分

26:00 multiply immediately. Instead, we want to group these like this. So every consecutive two elements,
立即乘以数。相反,我们要像这样组合这些元素。 所以每连续两个元素,

26:09 one, two and three and four and five and six and seven and eight, all of these should be now
1、2、3、4、5、6、6、7和8,所有这些现在都应该

26:13 basically flattened out and multiplied by a weight matrix. But all of these four groups here, we'd like
基本上平整成平整,乘以一个重量矩阵。但所有这四组,我们想

**weight** [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

**matrix**
矩阵 矩阵
数学与微积分

26:20 to process in parallel. So it's kind of like a batch dimension that we can introduce. And then we
平行地进行过程。所以它就像一个批量的维度, 我们可以引入。然后我们

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

26:26 can in parallel basically process all of these by Graham groups in the four batch dimensions
基本上可以平行地处理所有这些事情,由Graham集团在四批量中处理

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

26:34 of an individual example, and also over the actual batch dimension of the, you know, four
单个示例的批量尺寸,以及

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

26:39 examples in our example here. So let's see how that works. Effectively, what we want is right now,
举个例子,让我们看看这是怎么回事。实际上,我们现在想要的是,

26:45 we take a four by 80 and multiply by 80 by 200 to in the linear layer. This is what happens.
我们用一个四乘80,再乘80乘200到线性层。这就是发生的情况。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

26:53 But instead of what we want is we don't want 80 characters or 80 numbers to come in. We only
但是我们想要的不是 我们不想80个字符或80个数字进来

26:59 want two characters to come in on the very first layer. And those two characters should be fused.
想要在第一个层有两个字符。 这两个字符应该被连接 。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

27:03 So in other words, we just want 20 to come in, right? 20 numbers would come in. And here we
换句话说,我们只要20个数字,对吗?

27:11 don't want a four by 80 to feed into the linear layer. We actually want these groups of two to
我们其实想要的就是这组人

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

27:16 feed in. So instead of four by 80, we want this to be a four by four by 20. So these are the
输入 fef。所以,我们想要的不是四比八十,而是四比四 20。所以这就是

27:24 four groups of two. And each one of them is 10 dimensional vector. So what we want is now
4组2, 共四组。每组一个是10维矢量。 所以我们想要的是现在

**vector**
向量 向量
数学与微积分

27:30 is we need to change the flatten layer. So it doesn't output a four by 80, but it outputs a four
需要改变平面层。 因此它不会输出一个四乘80, 但是它会输出一个四

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

27:35 by four by 20, where basically these every two consecutive characters are packed in on the
将每两个连续字符加到

27:44 very last dimension. And then these four is the first batch dimension. And this four is the
也就是这四个是第一批的

**dimension**
维度 维度
数学与微积分

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

27:50 second batch dimension referring to the four groups inside every one of these examples.
第二部分涉及每个例子中四个组。

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

27:55 And then this will just multiply like this. So this is what we want to get to. So we're
然后,这样一来,我们就会成倍地增加。所以这就是我们要达到的目标。所以我们

28:00 going to have to change the linear layer in terms of how many inputs it expects. It shouldn't expect
将不得不改变线性层, 以它期望的输入量为单位。 它不应该期待

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

28:05 80, it should just expect 20 numbers. And we have to change our flatten layer. So it doesn't just
80,它应该会预计20个数字。我们必须改变平层。所以它不只是

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

28:10 fully flatten out this entire example. It needs to create a four by four by 20 instead of four by
完全平整整个示例。 它需要创建一个四乘四乘四x20,而不是四

28:16 80. So let's see how this could be implemented. Basically, right now we have an input that
80. 因此,让我们来看看如何实施。基本上,我们现在有一个投入

28:21 is a four by eight by 10 that feeds into the flatten layer. And currently the flatten layer
向平层进料。目前平层是平层

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

28:26 just stretches it out. So if you remember the implementation of flatten, it takes our x and
所以如果你记得平板的安装,它需要我们的x和

**implementation**
实现 实现
代码与工程

28:33 it just views it as whatever the batch dimension is, and then negative one. So effectively what it
将它视为任何批量尺寸,然后是负的。所以有效什么

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

28:38 does right now is it does EW of four negative one. And the shape of this of course is four by 80.
现在的答案是,它确实以四个负1为EW。当然这个形状是四比八十。

28:45 So that's what currently happens. And we instead want this to be a four by four by 20,
这就是目前发生的情况。我们更希望这个是 20比4,4

28:50 where these consecutive 10 dimensional vectors get concatenated. So you know how when Python
连续10维矢量被连接的位置。你知道当 Python 时如何

28:56 you can take a list of range of 10. So we have numbers from zero to nine. And we can index like
你可以列出一个10范围范围的列表。我们从零到九个数字, 我们可以用索引来显示

29:04 this to get all the even parts. And we can also index like starting at one and going the steps of
来获得所有偶数部分。我们也可以像开始一个步骤,

29:09 two to get all the odd parts. So one way to implement this, it would be as follows, we can
来得到所有奇特的部分。所以,一个方法来实施这个, 它会如下: 我们可以

29:16 take E and we can index into it for all the batch elements. And then just even elements in
将 E 取入, 我们可以对所有批量元素进行索引。 然后甚至将元素输入

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

29:23 this dimension. So at indexes zero, two, four and eight. And then all the parts here from this last
。 所以在指数0, 2, 4和8, 然后从这里的末端

**dimension**
维度 维度
数学与微积分

29:31 dimension. And this gives us the even characters. And then here, this gives us all the odd characters.
维度。这让我们有了偶数字符。然后,这里,这个给了我们所有奇数字符。

**dimension**
维度 维度
数学与微积分

29:41 And basically what we want to do is we want to make sure that these get concatenated
我们想要做的 基本上是确保这些被吸收

29:45 in PyTorch. And then we want to concatenate these two tensors along the second dimension.
然后我们想把这两组 10个分母结合到第二个维度上

**pytorch**
PyTorch PyTorch
代码与工程

**dimension**
维度 维度
数学与微积分

29:51 So this and the shape of it would be four by four by 20. This is definitely the result we want. We are
所以这个形状和它的形状 将是四乘四乘四x二十, 这肯定是我们想要的结果。我们是

29:57 explicitly grabbing the even parts and the odd parts. And we're arranging those four by four by 10
明确抓住偶数部分和奇数部分。我们正在安排这四个四个乘四个乘十个

30:04 right next to each other and concatenate. So this works. But it turns out that what also works
彼此相邻并彼此交融。所以这很有效,但结果发现,那也有效

30:10 is you can simply use a view again and just request the right shape. And it just so happens
就可以简单地再次使用一个视图, 要求正确的形状。 它就是这样发生的

30:17 that in this case, those vectors will again end up being arranged exactly the way we want.
如此一来 这些矢量最终会按照我们想要的方式 被安排好

30:23 So in particular, if we take E and we just view it as a four by four by 20, which is what we want,
特别是如果我们拿E,我们只要把它看作是一个四乘四乘二十, 这就是我们想要的,

30:28 we can check that this is exactly equal to what let me call this, this is the explicit
我们可以检查这是否和 我所说的完全相等,这是明确的

30:33 concatenation, I suppose. So explicit that shape is four by four by 20. If you just view it as
很明显,形状是四乘四乘二十

30:41 four by four by 20, you can check that when you compare to explicit, you got a big, this is
4乘4乘4 4乘四 20乘20 你可以检查 当你比较清晰时

30:47 element wise operation. So making sure that all of them are true, evaluates to true. So basically,
智慧的操作。 所以,确保它们都是真实的, 评估它们是否真实。 因此,基本上,

**operation**
运算 运算
神经网络基础

30:53 long story short, we don't need to make an explicit call to concatenate, etc. We can simply take
长长的短长的短话,我们不需要明确呼吁进行交融等等。我们可以简单地采取

30:58 this input tensor to flatten and we can just view it in whatever way we want. And
输入的电压会加速平坦, 我们可以以任何我们想要的方式看待它。

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

31:05 in particular, we don't want to stretch things out with negative one. We want to
尤其是,我们不想用负面的来拉开事情。

31:09 actually create a three dimensional array. And depending on how many vectors that are consecutive,
实际创建一个三维矩阵。取决于有多少矢量是连续的,

31:15 we want to fuse like for example, two, then we can just simply ask for this dimension to be 20.
我们想像,比如说,二, 发火,然后我们就可以简单地要求这个维度达到20。

**dimension**
维度 维度
数学与微积分

31:22 And use a negative one here. And by tort will figure out how many groups it needs to pack
并使用负的。通过侵权行为,可以找出需要打包多少组

31:27 into this additional batch dimension. So let's now go into flatten and implement this.
进入额外的批量维度。让我们现在来平整,执行这个。

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

31:33 Okay, so I scrolled up here to flatten. And what we'd like to do is we'd like to
好吧,我把这里卷起来 平整。我们想做的是

31:37 change it now. So let me create a constructor and take the number of elements that are consecutive
现在改变它。所以让我创建一个构建器, 并选择连续的元素数

31:42 that we would like to concatenate now in the last dimension of the output.
并且我们希望现在将我们放在产出的最后方面。

**dimension**
维度 维度
数学与微积分

31:46 So here, we're just going to remember, solve that n equals n. And then I want to be careful here
所以,在这里,我们只是要记住, 解决n等于n。然后我想小心这里

31:52 because by pytorch actually has a torched up flatten and its keyword arguments are different.
因为烧焦机实际上有一把火炬 平坦,关键词参数是不同的。

**pytorch**
PyTorch PyTorch
代码与工程

31:57 And they kind of like function differently. So our flatten is going to start to depart
并且他们有点喜欢不同功能。所以我们的公寓要开始起飞了

32:01 from pytorch flatten. So let me call it flat flatten consecutive or something like that,
直接从平板上打出来,我把它称为平坦的连续10个或类似的东西

**pytorch**
PyTorch PyTorch
代码与工程

32:06 just to make sure that our APIs are about equal. So this basically flattens only some
只是为了确保我们的APPs是平等的。所以这基本上只平了一部分

32:13 n consecutive elements and puts them into the last dimension.
n 连续元素,并将其置于最后一层。

**dimension**
维度 维度
数学与微积分

32:17 Now here, the shape of x is B by T by C. So let me pop those out into variables
现在, x 的形状是 B 乘 T 乘 T C 。 所以让我把这些放入变量

32:25 and recall that in our example down below, B was four, T was eight and C was 10.
并记得在下面的例子中,B是4岁,T是8岁,C是10岁。

**recall** [ri'kɔ:l]
召回率 一种分类模型 指标，用于回答以下问题：在所有可能的正类别标签中， 模型正确地识别出了多少个？即：
神经网络基础

32:33 Now instead of doing x dot view of B by negative one,
现在不用用负一对B做x点视图

32:39 right, this is what we had before, we want this to be B by negative one by.
没错,这是我们以前拥有的 我们希望这个是B乘负一个

32:49 And basically here, we want C times n. That's how many consecutive elements we want.
基本上这里,我们需要C乘以n。 这就是我们想要的连续元素数量。

32:56 And here instead of negative one, I don't super love the use of negative one because
而不是消极的,我不喜欢使用负面的 因为

33:00 I like to be very explicit so that you get error messages when things don't go according
我希望非常明确,这样你就可以在事情不发生时得到错误信息

33:04 to your expectation. So what do we expect here? We expect this to become T divide and
我们期待你们的期望是什么?我们在这里期望什么?我们期待这变成T分裂,

33:10 using integer division here. So that's what I expect to happen.
使用整数分法,所以我期望会这样

33:14 And then one more thing I want to do here is remember previously all the way in the beginning,
还有一件事我想在这里做的 就是记住以前 最开始的时候

## 神经网络 · 张量 33:18

33:18 n was three. And basically we're concatenating all the three characters that existed there.
基本上我们把这里的三个人物 全部分解出来

33:26 So we basically are concatenated everything. And so sometimes I can create a spurious dimension
所以基本上我们把一切都集中在一起。所以有时我也可以制造一个虚假的维度

**dimension**
维度 维度
数学与微积分

33:31 of one here. So if it is the case that x dot shape at one is one, then it's kind of like a spurious
此处一个。所以如果一个X点形状是一,那就有点像假的

33:38 dimension. So we don't want to return a three dimensional tensor with a one here, we just
所以我们不想返回三维抗拉 与一个三维抗压在此,我们只是

**dimension**
维度 维度
数学与微积分

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

33:44 want to return it to dimensional tensor exactly as we did before. So in this case, basically we
想要和以前一样 返回到维度抗拉。所以在这种情况下,基本上我们

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

33:50 will just say x equals x dot squeeze, that is a pytorch function. And squeeze takes a dimension
将只表示 x x 等于 x点挤压, 这是一个烧火机函数。 挤压需要一个尺寸

**pytorch**
PyTorch PyTorch
代码与工程

**dimension**
维度 维度
数学与微积分

33:59 that it either squeezes out all the dimensions of a tensor that are one, or you can specify the
或可以指定

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

34:05 exact dimension that you want to be squeezed. And again, I like to be as explicit as possible
精确的尺寸,您想要被挤压。再说一遍,我想尽可能清晰

**dimension**
维度 维度
数学与微积分

34:11 always. So I expect to squeeze out the first dimension only of this tensor,
所以我希望只挤出这个阵列的第一个维度

**dimension**
维度 维度
数学与微积分

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

34:17 this three dimensional tensor. And if this dimension here is one, then I just want to
如果这个维度是1,那么我只想

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**dimension**
维度 维度
数学与微积分

34:21 return b by c times n. And so self dot out will be x. And then we return self dot out.
b 返回 c乘以 n。 所以自我点会是 x。 然后我们返回 自己点出来。

34:28 So that's the candidate implementation. And of course, this should be self dot in
这就是候选人的完成。当然,这应该是自我点

**implementation**
实现 实现
代码与工程

34:33 instead of just n. So let's run. And let's come here now. And take it for a spin. So flatten
而不是只是n. 所以让我们跑吧。我们现在就来,然后把它转一圈。所以平

34:41 consecutive. And in the beginning, let's just use eight. So this should recover the previous
。在开始时,我们只需使用 8。所以这个应该恢复上一个

34:49 behavior. So flag consecutive of eight, which is the current block size, you can do this,
。所以连续8个的旗帜,也就是当前区块大小, 你可以做到这一点,

34:57 that should recover the previous behavior. So we should be able to run the model.
恢复以前的行为。所以我们应该能够运行模型。

35:02 And here we can inspect, I have a little code snippet here, where I iterate over all the layers,
在这里我们可以检查,我这里有一个小代码片段, 我在那里横跨所有层层,

35:08 I print the name of this class and the shape. And so we see the shapes as we expect them after
我打印了这个类的名称和形状。 因此我们可以看到这些形状, 因为我们期待它们之后

35:18 every single layer in its output. So now let's try to restructure it using our flatten
每一层的输出。 所以现在让我们尝试用平板来调整它

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
> 神经网络基础

35:23 consecutive and do it hierarchically. So in particular, we want to flatten consecutive,
并且按等级顺序进行。所以我们想连续进行

35:29 not just not block size, but just two. And then we want to process this with linear. Now,
不仅不是块大小,而是两个。然后我们要用线性处理这个。现在,

35:35 then the number of inputs to this linear will not be an embed times block size,
这样, 线性输入的数量就不会是一个嵌入时间块大小,

35:39 it will now only be an embed times 20. This goes through the first layer. And now we can,
现在只能是嵌入20乘以20 穿过第一层

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
> 神经网络基础

35:47 in principle, just copy paste this. Now the next linear layer should expect n hidden times two.
原则上,只需复制粘贴这个。 现在下一个线性层应该会期望隐藏的乘以 2 。

> **layer** ['leiə]
> 层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
> 神经网络基础

35:54 And the last piece of it should expect an hidden times two again.
最后一部分应该会再次 隐藏两次

36:01 So this is sort of like the naive version of it. So running this, we now have a much,
所以这有点像天真版。所以运行这个,我们现在有很多,

36:07 much bigger model. And we should be able to basically just forward the model.
更大规模的模型。我们应该能够基本上 推进模型

36:13 And now we can inspect the numbers in between. So four by 20 was flatten consecutive into
现在我们可以检查中间之间的数字。所以4比20 连续连续进入

36:21 four by 20. This was projected into four by four by 200. And then Bashorm just worked out of the box.
预测到4乘4乘200 然后Bashorm刚从盒子里出来

36:30 And we have to verify that Bashorm does the correct thing, even though it takes three
我们必须核实巴索姆做的正确 尽管需要三个

36:33 dimensional impedance instead of two dimensional input. Then we have 10h, which is element wise.
维度阻力而不是二维输入。然后我们有10小时,这是明智的元素。

36:38 Then we crushed it again. So we flatten consecutively and ended up with a four by two by
然后我们又把它压碎了。我们连续地压扁它,然后,我们用一个四乘二击倒它。

36:43 400 now. Then linear brought it back down to 200. Bashorm 10h. And lastly, we get a four by
直线将它带回200,10小时,最后我们得到4

36:50 400. And we see that the flatten consecutive for the last flatten here, it squeezed out that dimension
400. 我们看到这里最后一平坦的连续倒数 挤出那个维度

**dimension**
维度 维度
数学与微积分

36:56 of one. So we only ended up with four by 400. And then linear Bashorm 10h and the last linear
以一比一。所以我们最后只能用四乘400。然后是线性Basshorm 10,最后的线性

37:03 layer to get our logits. And so the logits end up in the same shape as they were before.
以获得我们的登录。 因此, 日志最终会以和以前相同的形状结束 。

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

37:08 But now we actually have a nice three layer neural net. And it basically corresponds to,
但现在我们实际上有一个良好的三层神经网。它基本上相当于,

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

**neural net**
神经网络 神经网络
神经网络基础

37:13 oops, sorry, it basically corresponds exactly to this network now, except only this piece
抱歉,抱歉,它基本上和现在的网络完全吻合 除了这个

37:19 here because we only have three layers. Whereas here in this example, there's four layers,
因为我们只有三层。这里的例子有四层,

37:24 with a total receptive field size of 16 characters instead of just eight characters.
场面总可容纳大小为16个字符,而不是8个字符。

37:30 So the block size here is 16. So this piece of it is basically implemented here.
块大小是16,所以这里基本上就是执行

37:36 Now we just have to kind of figure out some good channel numbers to use here.
现在我们只需要找出一些好的频道号码 来使用这里

37:41 Now in particular, I changed the number of hidden units to be 68 in this architecture,
现在,我特别把这个建筑中的隐藏单位数目 改为68个

37:46 because when I use 68, the number of parameters comes out to be 22,000. So that's exactly the
因为当我用68,参数数是22000。所以这就是

## 批次大小 · 神经网络 37:51

37:51 same that we had before. And we have the same amount of capacity at this neural net in terms of
和以前一样。我们在这个神经网里拥有同样数量的能量

**neural net**
神经网络 神经网络
神经网络基础

37:55 the number of parameters. But the question is whether we are utilizing those parameters in a
参数数量。但问题是,我们是否在

37:59 more efficient architecture. So what I did then is I got rid of a lot of the debugging cells here,
更有效率的建筑。我当时所做的是, 我摆脱了很多调试细胞在这里,

38:05 and I re-rend the optimization and scrolling down to the result, we see that we get
我重新恢复优化 并滚到结果, 我们可以看到我们得到了

38:10 the identical performance roughly. So our validation loss now is 2.029, and previously
大致相同性能。所以我们现在的验证损失是2.029, 先前

**validation**
验证 验证
训练与优化

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

38:16 it was 2.027. So controlling for the number of parameters, changing from the flag to hierarchical
控制参数数量,从国旗到等级

38:21 is not giving us anything yet. That said, there are two things to point out. Number one,
还没有给我们任何信息,但有两点要指出。

38:27 we didn't really torture the architecture here very much. This is just my first guess.
我们没有在这里折磨过建筑,这是我第一次猜测

38:31 And there's a bunch of hyperparameters searched that we could do in terms of how we allocate
有很多超参数搜索, 我们可以做一些关于我们如何分配

38:36 our budget of parameters to what layers. Number two, we still may have a bug inside the Bash
第二,我们可能还在巴什有窃听器

38:42 Trorm 1D layer. So let's take a look at that because it runs, but does it do the right thing?
托罗姆 1D层。让我们看看它,因为它运行着, 但是它做对了吗?

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

38:50 So I pulled up the layer inspector, sort of that we have here, and printed out the shape along
于是我拉起层检查员 大致上我们这里的 并打印了形状

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

38:55 the way. And currently it looks like the Bash Trorm is receiving an input that is 32x4x68.
。目前它看起来像巴什特罗姆正在接收32x4x68的输入。

39:02 And here on the right, I have the current implementation of Bash Trorm that we have
右边,我这里有我们目前执行的巴什特罗姆

**implementation**
实现 实现
代码与工程

39:05 right now. Now this Bash Trorm assumed in the way we wrote it, and at the time, that x is two-dimensional.
现在,这个巴什特罗姆 假设我们写的方式,当时,x是二维的。

39:12 So it was n by d, where n was the batch size. So that's why we only reduced the mean and
所以是n by d, n 是哪里的批量大小。 所以我们只减少了平均值,

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

39:18 the variance over the 0th dimension. But now x will basically become three-dimensional.
与0维值的差数。但现在x将基本上变成三维。

**dimension**
维度 维度
数学与微积分

39:23 So what's happening inside the Bash Trorm layer right now, and how come it's working at all
所以现在巴什特罗姆层内部发生了什么 以及它是如何运作的

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

39:26 and not giving any errors? The reason for that is basically because everything broadcasts properly,
而不是错误。原因就在于所有节目都正常播放

39:31 but the Bash Trorm is not doing what we want it to do. So in particular, let's basically think
特别是,我们基本上想想

39:37 through what's happening inside the Bash Trorm. Looking at what's happening here.
看着这里发生的一切

39:43 I have the code here. So we're receiving an input of 32x4x68. And then we are doing
代码在这里,我们收到32x4x68的输入

39:51 here x.mean, here I have e instead of x. But we're doing the mean over 0. And that's
这里 x. mean, 在这里我用e代替x。 但我们的平均值超过0。

39:57 actually giving us 1x4x68. So we're doing the mean only over the very first dimension.
实际上我们给了1x4x68。所以我们只做了第一个维度的平均值。

**dimension**
维度 维度
数学与微积分

40:02 And it's giving us a mean and a variance that still maintain this dimension here. So these
给我们一个手段和差异 维持了这个维度

**dimension**
维度 维度
数学与微积分

40:08 means are only taking over 32 numbers in the first dimension. And then when we perform this,
意思是第一个维度中只接收32个以上的数字。 当我们执行这个时,

**dimension**
维度 维度
数学与微积分

40:14 everything broadcasts correctly still. But basically what ends up happening is
所有的广播都正确无误地播放。但基本上,最终的结果是:

40:20 when we also look at the running mean, the shape of it. So I'm looking at the model
当我们同时看运行的暗值,它的形状。 所以我在看模型

40:27 that layers at 3D, which is the first Bash Trorm layer, and then looking at whatever the running
3D的三维层,是第一个BashTrorm层,然后看任何运行的

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

40:31 mean became and its shape. The shape of this running mean now is 1x4x68.
正向正态开始, 形状也开始。 此运行平均值的形状是 1x4x68 。

40:38 Instead of it being just a size of dimension, because we have 68 channels, we expect to have
与其仅仅是一个尺寸大小的维度,不如说它是一个规模的维度。

**dimension**
维度 维度
数学与微积分

40:45 68 means and variances that we're maintaining. But actually we have an array of 4x68. And so
和我们所保持的68种方式和差异。但实际上我们有4x68的阵列。所以

40:51 basically what this is telling us is this Bash Trorm is currently working in parallel over
基本上,这告诉我们的是 这个巴什特罗姆目前同时工作

41:01 4x68 instead of just 68 channels. So basically we are maintaining statistics for every one of these
基本上我们维持着每一个频道的统计

## 批归一化 · 批次大小 41:10

41:10 four positions individually and independently. And instead what we want to do is we want to treat
四个立场,单独和独立地

41:15 this 4 as a Bash dimension just like the 0th dimension. So as far as the Bash Trorm is
4是像0维一样的巴什次元。直到巴什特罗姆

**dimension**
维度 维度
数学与微积分

41:22 concerned, it doesn't want to average, we don't want to average over 32 numbers. We want to
相关,它不想平均,我们不想平均超过32个数字。

41:26 now average over 32 times four numbers for every single one of these 68 channels. And so let me now
现在平均超过32乘以4的 数字,68个频道中每个频道 平均有32个

41:34 remove this. It turns out that when you look at the documentation of tors.min,
此处删除。 结果发现当您查看 torrs.min 的文档时,

41:42 so let's go to tors.min, in one of its signatures, when we specify the dimension,
所以让我们去用托雷斯.min, 在其一个签名中, 当我们指定维度时,

**dimension**
维度 维度
数学与微积分

41:53 we see that the dimension here is not, it can be int or it can also be a tuple of ints.
我们发现这里的维度不是,它可以是内含的,也可以是一小块内含的。

**dimension**
维度 维度
数学与微积分

41:58 So we can reduce over multiple integers at the same time, over multiple dimensions at the same time.
这样我们就可以同时减少多个整数, 同时减少多个维度。

42:03 So instead of just reducing over 0, we can pass in a tuple, 0, 1, and here 0, 1 as well.
所以,我们不用减少0, 而不是仅仅减少0, 我们可以通过一个图普,0,1, 这里,0, 1,以及。

42:10 And then what's going to happen is the output, of course, is going to be the same. But now what's
然后接下来的将是产出,当然, 将会是相同的。但现在

42:14 going to happen is because we reduce over 0 and 1, if we look at in mean dot shape, we see that
因为我们减少0和1, 如果我们看中点形状,我们可以看到

42:20 now we've reduced, we took the mean over both the 0th and the first dimension. So we're just
现在我们缩小了,我们采取了平均值 超过0和1维。所以我们只是

**dimension**
维度 维度
数学与微积分

42:27 getting 68 numbers and a bunch of spurious dimensions here. So now this becomes 1 by 1 by 68,
这儿有68个数字和一堆虚假的维度。现在,这个数字变成1乘1乘1比68,

42:33 and the running mean and the running variance analogously will become 1 by 1 by 68. So even
和运行平均值和运行差异 类似地,将变成1乘1乘1等于68。 所以,甚至算起来

42:39 though there are the spurious dimensions, the current, the correct thing will happen in that
虽然有虚假的维度, 时空,正确的事情会发生

42:43 we are only maintaining means and variances for 64, sorry, for 68 channels. And we're now
我们只维持了64个频道的 资源和差异

42:50 calculating the mean variance across 32 times four dimensions. So that's exactly what we want.
计算出32乘以4维的平均值差。这就是我们想要的。

42:56 And let's change the implementation of BatchNorm 1D that we have so that it can
让我们改变我们拥有的批量Norm1D的实施, 以便它能够

**implementation**
实现 实现
代码与工程

**batchnorm**
批归一化 批归一化
训练与优化

43:01 take in two dimensional or three dimensional inputs and perform accordingly.
以二维或三维输入并相应进行。

43:05 So at the end of the day, the fix is relatively straightforward. Basically, the dimension we want
所以在一天的结尾, 修正是相对简单的。基本上, 我们想要的维度

**dimension**
维度 维度
数学与微积分

43:09 to reduce over is either 0 or the tuple 0 and 1, depending on the dimensionality of X.
减少值为 0 或 tupl0和 1,视X 的维度而定。

43:15 So if X dot and M is 2, so it's a two dimensional tensor, then the dimension we want to reduce
所以如果X点和M是 2,那么它是一个二维的抗拉, 那么我们想要缩小的维度

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**dimension**
维度 维度
数学与微积分

43:20 over is just the integer 0. L of X dot and M is 3, so it's a three dimensional tensor,
以上是整数 0。L X点和M是3, 所以它是一个三维的抗拉,

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

43:25 then the dims we're going to assume are 0 and 1 that we want to reduce over. And then here we just
然后我们假设的暗淡是0和1 我们要减少零和一。然后我们在这里

43:32 pass in dim. And if the dimensionality of X is anything else, we'll now get an error, which is good.
如果X的维度是其它的东西,我们现在会发现一个错误,这是很好的。

43:39 So that should be the fix. Now I want to point out one more thing. We're actually
应该是解决问题的办法。现在我想指出一件事,我们实际上是

43:42 departing from the API of PyTorch here a little bit, because when you come to BatchNorm 1D
离开皮托尔奇的API 这里一点点, 因为当你来到 BatchNorm1D

**pytorch**
PyTorch PyTorch
代码与工程

**batchnorm**
批归一化 批归一化
训练与优化

43:47 in PyTorch, you can scroll down and you can see that the input to this layer can either be
在 PyTorch 中, 您可以滚动向下, 您可以看到此层的输入可以是

**pytorch**
PyTorch PyTorch
代码与工程

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

43:52 N by C, where N is the batch size and C is the number of features or channels, or it actually
N . N 的N . . , N 是批量大小, C是特性或渠道的数量, 或者说它实际上

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

43:57 does accept three dimensional inputs, but it expects it to be N by C by L, where L is,
接受三维输入, 但预计是NxCxL, L是,

44:03 say, like the sequence length or something like that. So this is a problem because you see how
比如,像序列长度之类的。所以这是一个问题,因为你看到

44:09 C is nested here in the middle. And so when it gets three dimensional inputs, this BatchNorm
C 嵌巢在中间。 当它得到三维输入时, 这个批次诺姆

**batchnorm**
批归一化 批归一化
训练与优化

44:14 layer will reduce over 0 and 2 instead of 0 and 1. So basically PyTorch BatchNorm 1D layer assumes that
将减少 0和2 的层, 而不是 0和 1。 所以基本上 PyTorrch BatchNorm1D 层假定

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

**pytorch**
PyTorch PyTorch
代码与工程

44:23 C will always be the first dimension, whereas we assume here that C is the last dimension,
C永远是第一个层面,而我们在这里假定C是最后一个层面,

**dimension**
维度 维度
数学与微积分

44:30 and there are some number of batch dimensions beforehand. And so it expects N by C or N by C
并且事先有一些批量尺寸。所以它预期N 以C或N 以 C 表示

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

44:38 by L, we expect N by C or N by L by C. And so it's a deviation. I think it's okay. I prefer it this
L,我们预期NC或NNLC。所以这是一个偏差。我想没关系。我更喜欢这个

44:48 way honestly, so this is the way that we will keep it for our purposes. So I redefine the layers,
诚实地说,这就是我们保持它的方式。所以我重新定义了层次,

44:53 reinitialize the neural nut, and then a single forward pass with a break just for one step.
重新激活神经螺旋 然后一个前行 休息一步

**forward pass**
前向传播 前向传播
神经网络基础

44:58 Looking at the shapes along the way, they're of course identical, all the shapes are the same,
一路看形状,它们当然是相同的, 所有形状都是相同的,

45:02 but the way we see that things are actually working as we want them to now is that when
但是我们如何看待事情实际上正在按照我们现在想要的那样运转,

45:06 we look at the BatchNorm layer, the running mean shape is now 1 by 1 by 68. So we're only
我们看看批次诺姆层, 运行的平均值形状现在是1乘1乘1等于68。 所以我们只是

**batchnorm**
批归一化 批归一化
训练与优化

**layer** ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

45:11 maintaining 68 means for every one of our channels, and we're treating both the 0th and the first
我们每个频道都维持68种通讯设备, 我们正在处理第1和第2个

45:16 dimension as a batch dimension, which is exactly what we want. So let me retrain the neural nut
成批量维度,这正是我们想要的。所以让我重新训练神经神经元

**dimension**
维度 维度
数学与微积分

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

45:21 now. Okay, so I retrained the neural nut with the bug fix, we get a nice curve. And when we look
现在,我用补虫器重新训练神经螺母 我们得到一个不错的曲线

## 批归一化 · 验证 45:25

45:25 at the validation performance, we do actually see a slight improvement. So it went from 2.029 to
验证性能的测试,我们确实看到有一点点改善。所以从2.029到

**validation**
验证 验证
训练与优化

45:29 2.022. So basically the bug inside the BatchNorm was holding us back like a little bit, it
2.022。 所以基本上说,在BatchNorm的虫子 把我们拖得有点紧

**batchnorm**
批归一化 批归一化
训练与优化

45:35 looks like. And we are getting a tiny improvement now, but it's not clear if this is statistically
看上去像。我们现在正在取得一个小改进, 但不清楚这是否统计上

45:39 significant. And the reason we slightly expect an improvement is because we're not maintaining
因为我们没有保持

45:46 so many different means and variances that are only estimated using 32 numbers effectively.
如此多不同的手段和差异,仅以32个数字有效估算。

45:51 Now we are estimating them using 32 times 4 numbers. So you just have a lot more numbers
现在我们用32乘以4的数来估计它们。所以你只有更多数字

45:56 that go into any one estimate of the mean and variance. And it allows things to be a bit
以任何一种估计平均值和差值。它允许事物稍稍变小

46:01 more stable and less wiggly inside those estimates of those statistics. So pretty nice.
这些数据的估计数内更稳定,更不那么虚张声势。

46:07 With this more general architecture in place, we are now set up to push the performance further
随着这一更总体的结构的建立,我们现在成立的目的是进一步推进业绩

46:11 by increasing the size of the network. So for example, I bumped up the number of embeddings
增加网络的大小。例如,我增加了嵌入的数量

**embedding** [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

46:16 to 24 instead of 10, and also increased number of hidden units. But using the exact same architecture,
将它变为24个而不是10个,并增加隐藏单位的数量。但使用相同的结构,

46:22 we now have 76,000 parameters. And the training takes a lot longer. But we do get a nice curve.
训练需要更长的时间 但我们的曲线不错

**training** ['treinin]
训练 确定构成模型的理想参数 的过程。
神经网络基础

46:28 And then when you actually evaluate the performance, we are now getting validation
当你实际评估业绩时, 我们现在正在得到验证

**validation**
验证 验证
训练与优化

46:31 performance of 1.993. So we've crossed over the 2.0 territory. And we're at about 1.99.
超过2.9度,我们正值1.99

46:39 But we are starting to have to wait quite a bit longer. And we're a little bit in the dark
但我们开始要等更久一点了。

46:44 with respect to the correct setting of the hyperparameters here and the learning rates
此处超参数正确设置和学习率

46:47 and so on, because the experiments are starting to take longer to train. And so we are missing
等,因为实验开始需要更长的时间来训练。所以我们缺少了

46:51 sort of like an experimental harness on which we could run a number of experiments
类似实验性利用, 我们可以运行一些实验

46:56 and really tune this architecture very well. So I'd like to conclude now with a few notes.
并且非常地调整这个建筑。所以我想用几个笔记来结束现在的演讲。

47:00 We basically improved our performance from a starting of 2.1 down to 1.9. But I don't want
我们基本上提高了我们的绩效,从2.1的开始 下到1.9。

47:06 that to be the focus because honestly, we're kind of in the dark. We have no experimental harness.
重点是说实话,我们是在黑暗中 我们没有实验性工具

47:10 We're just guessing and checking. And this whole thing is terrible. We're just looking
我们只是猜一猜,看看 整个事情太可怕了

47:13 at the training loss. Normally, you want to look at both the training and the validation loss
通常,您既想查看培训,也想查看审定损失。

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

**loss** [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

47:18 together. The whole thing looks different if you're actually trying to squeeze out numbers.
两者加在一起,如果你们想挤出数字,整个过程看起来就不一样了。

47:22 That said, we did implement this architecture from the WaveNet paper, but we did not implement this
虽然如此,我们确实从WaveNet文件中执行了这一架构,但我们没有执行

47:29 specific forward pass of it where you have a more complicated linear layer sort of that is this gated
具体前方的过路, 在那里您有一个更复杂的线性层, 类似这个门

**forward pass**
前向传播 前向传播
神经网络基础

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

47:36 linear layer kind of. And there's residual connections and skip connections and so on.
线性层。还有剩余连接,跳过连接等等。

**layer** ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

47:41 So we did not implement that. We just implemented this structure. I would like to briefly hint
因此,我们没有执行。 我们刚刚实施了这一结构。

47:46 or preview how what we've done here relates to convolutional neural networks as used in the
或者预览我们在这里所做的 与在

47:51 WaveNet paper. And basically, the use of convolution is strictly for efficiency. It doesn't actually
基本上,使用电波网络纸。 基本上, 利用变化是严格为了效率。它实际上并不

**convolution** [ˌkɒnvə'luːʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

## 训练集 · 神经网络 47:56

47:56 change the model we've implemented. So here, for example, let me look at a specific name to work
更改了我们实施的模型。例如,这里,让我看看一个具体的名称来工作

48:02 with an example. So there's a name in our training set and it's Deondre. And it has seven letters.
举个例子。所以我们训练组里有一个名字,是迪昂德雷,它有七个字母

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

48:08 So that is eight independent examples in our model. So all these rows here are independent
这就是我们模型中的八个独立的例子。所以所有这行都是独立的

48:14 examples of Deondre. Now you can forward, of course, any one of these rows independently.
迪昂德雷的例子。现在你可以独立地向前推进任何一行了。

48:19 So I can take my model and call it on any individual index. Notice by the way here,
所以我可以把我的模型 调用任何个人索引

48:26 I'm being a little bit tricky. The reason for this is that extra at seven dot shape is just
我有点狡猾,原因就是 7点外加的

48:33 one dimensional array of eight. So you can't actually call the model on it. You're going to
以一维数为八。 所以你不能将模型调用在它上, 你会

48:38 get an error because there's no batch dimension. So when you do extra at
错误会发生错误, 因为没有批量尺寸。 所以当您在

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

48:44 list of seven, then the shape of this becomes one by eight. So I get an extra batch dimension of one,
列表中的七位, 然后这个形状变成一个乘以八。 所以我得到一个额外的批量尺寸

**batch** [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

**dimension**
维度 维度
数学与微积分

48:50 and then we can forward the model. So that forwards a single example. And you might imagine that
you
然后我们可以推出模型。这样就举了一个例子。你就可以想象

48:57 actually may want to forward all of these eight at the same time. So pre allocating some memory
可能想要同时将全部八点同时转发。 所以先分配一些内存

49:04 and then doing a for loop eight times and forwarding all of those eight here will give us
然后做八次循环 转发所有八个人

49:10 all the logits in all these different cases. Now for us with the model as we've implemented it right now,
现在,我们用模型来做模型,因为我们现在已经应用了它

**logits**
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

49:15 this is eight independent calls to our model. But what convolutions allow you to do is it
这是八次独立呼叫我们的模型。但革命允许你做的是

49:20 allow you to basically slide this model efficiently over the input sequence. And so this for loop
允许您基本上将这个模型在输入序列上有效滑动。所以这是循环

49:27 can be done not outside in Python, but inside of kernels in CUDA. And so this for loop gets
可以在皮顿城外做,但可在CUDA的内核内部完成。因此,循环获得

49:33 hidden into the convolution. So the convolution basically you can think of it as it's a for
所以,这种混凝过程基本上 你可以把它当作是用来

**convolution** [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

49:38 loop, applying a little linear filter over space of some input sequence. And in our case, the space
循环,在输入序列空间上应用一个小线性过滤器。在我们的情况下,

49:45 we're interested in is one dimensional. And we're interested in sliding these filters over the input
我们感兴趣的是一维。我们有兴趣 将这些过滤器滑过输入

49:49 data. So this diagram actually is fairly good as well. Basically, what we've done is here they
数据。所以这个图表其实也相当不错。基本上,我们做的就是在这里

49:56 are highlighting in black one individual one single sort of like tree of this calculation.
以黑色显示单个的单种树 来显示这个计算结果

50:02 So just calculating the single output example here. And so this is basically what we've implemented
所以在这里计算一个输出示例。 因此,这基本上是我们所执行的

50:09 here. We've implemented a single this black structure, we've implemented that and calculated
。 我们在这里执行了一个单一的黑黑结构, 我们执行了这个,计算了

50:14 a single output like a single example. But what convolutions allow you to do is it allows you
单一个输出,如一个示例。但革命允许你做的是它允许你

50:19 to take this black structure and kind of like slide it over the input sequence here and
来将这个黑色结构 和类似滑动的黑结构 放在输入序列中

50:25 calculate all of these orange outputs at the same time. Or here that corresponds to calculating
同时计算所有这些橙色输出。 或者这里对应于计算

50:32 all of these outputs off at all the positions of the Andre at the same time. And the reason
所有这些产出 都同时在安德烈的所有岗位上 被关闭。以及原因

50:40 that this is much more efficient is because number one, as I mentioned the for loop is inside the
效率高得多是因为第一, 正如我提到,循环是

50:44 CUDA kernels in the sliding. So that makes it efficient. But number two, notice the variable
CUDA 滑动内核, 从而使其效率高。 但第二, 注意变量

50:50 reuse here. For example, if we look at this circle, this node here, this node here is the
此处再重新使用。 例如, 如果我们查看此圆, 这个节点, 这里的节点就是

50:55 right child of this node, but it's also the left child of the node here. And so basically this node
这个节点的右子, 但它也是这里的节点的左子。 所以基本上这个节点

51:02 and its value is used twice. And so right now in this naive way, we'd have to recalculate it.
其价值被使用两次。现在,我们不得不用这种天真的方式重新计算它。

51:10 But here we are allowed to reuse it. So in the convolutional neural network, you think of these
但是这里我们被允许再利用它。所以在 革命神经网络中,你想想这些

**neural network**
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

51:15 linear layers that we have above as filters. And we take these filters and their linear filters
线性层,我们作为过滤器在上面有。我们把这些过滤器和它们的线性过滤器取下

51:21 and you slide them over input sequence. And we calculate the first layer and then the second layer
然后将它们滑过输入序列。我们计算第一层,然后第二层

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

51:26 and then third layer and then the output layer of the sandwich. And it's all done very efficiently
然后是第三层,然后是三明治的输出层。 并且这一切都非常高效地完成了

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

51:30 using these convolutions. So we're going to cover that in a future video. The second thing I hope
利用这些革命的变化。所以我们在未来的视频里会报道这一点。我希望第二件事

51:35 you took away from this video is you've seen me basically implement all of these layer
你从这段视频中拿走了... ...你见过我基本上 都用这些层

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

51:40 Lego building blocks or module building blocks. And I'm implementing them over here. And we've
乐高的建筑构件或模块构件。我在这里执行这些构件。我们

51:45 implemented a number of layers together. And we've also implementing these these
一起执行了很多层。我们也执行了这些

51:49 containers. And we've overall pytorchified our code quite a bit more. Now basically what we're
容器。我们把代码的全体烧化了,现在基本上

51:55 doing here is we're re implementing torched.nn, which is the neural networks library on top of
正在实施点火.nn, 这是神经网络图书馆

52:02 torched. tensor . And it looks very much like this, except it is much better because it's in pytorch
火烧的电压器,它看起来非常像这个, 只是因为它在燃烧炉里。

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

**pytorch**
PyTorch PyTorch
代码与工程

52:08 instead of jinkling my Jupyter notebook. So I think going forward, I will probably have
而不是用我的柔和机笔记本敲打。

52:13 considered us having unlocked torched.nn. We understand roughly what's in there,
认为我们解开火炬了 我们大致了解里面是什么

52:18 how these modules work, how they're nested, and what they're doing on top of torched. tensor .
这些模块是如何运作的,如何筑巢, 以及他们在火炬的电炉上做什么。

**tensor** ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

52:23 So hopefully we're just we'll just switch over and continue and start using torched.nn directly.
希望我们只是换个位置 继续使用点火. nn

52:28 The next thing I hope you got a bit of a sense of is what the development process of building
接下来我希望你对 发展过程的建设有一定的认识

52:32 deep neural networks looks like, which I think was relatively representative to some extent.
深神经网络看起来很像, 我认为在某种程度上相对而言是具有代表性的。

52:37 So number one, we are spending a lot of time in the documentation page of pytorch. And we're
所以首先,我们花了很多时间 在皮托切的文献资料页上。

**pytorch**
PyTorch PyTorch
代码与工程

52:43 reading through all the layers looking at the documentations, what are the shapes of the
查看文档,读取所有层次的阅读,

52:47 inputs, what can they be, what does the layer do, and so on. Unfortunately, I have to say the
输入,它们能是什么, 一层会做什么,等等。 不幸的是,我不得不说

**layer** ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

52:52 pytorch documentation is not very good. They spend a ton of time on hardcore engineering of
皮火文件并不很好,他们花大量时间在核心工程上

**pytorch**
PyTorch PyTorch
代码与工程

52:58 all kinds of distributed primitives, etc. But as far as I can tell, no one is meeting the
各种分布的原始生物等等。

53:03 documentation, it will lie to you, it will be wrong, it will be incomplete, it will be unclear.
文献资料,它会骗你, 它将是错误的, 它将是不完整的,它将是模糊的。

53:10 So unfortunately, it is what it is. And you just kind of do your best
所以不幸的是,事情就是这样的。你只是尽力

53:13 with what they've given us. Number two, the other thing that I hope you got a sense of is,
第二,我希望你还有感想

53:23 there's a ton of trying to make the shapes work. And there's a lot of gymnastics around these
有很多的体操在周围

53:27 multi dimensional arrays. And are they two dimensional, three dimensional, four dimensional?
多元数组。它们是两个维,三个维,四个维吗?

53:31 What layers take what shapes? Is it NCL or NLC? And you're permuting and viewing. And
哪些层的形状?是NCL还是NLC?

## 训练 · 验证 53:38

53:38 it's just going to get pretty messy. And so that brings me to number three. I very often
这会变得乱得要命,所以我就会成为第三了

53:42 prototype these layers and implementations in Jupyter notebooks and make sure that all the
和在Jupyter笔记本中的执行,并确保

53:46 shapes work out. And I'm spending a lot of time basically babysitting the shapes and making sure
我花了很多时间 基本上照顾这些形状 确保

53:51 everything is correct. And then once I'm satisfied with the functionality in a Jupyter notebook,
完全正确,一旦我对Jupyter笔记本的功能满意

53:55 I will take that code and copy paste it into my repository of actual code that I'm training
我会把代码 复制并粘贴到我的 实际代码库里 我训练的

**training** ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

54:00 with. And so then I'm working with VS code on the side. So I usually have Jupyter notebook
和。然后我用VS代码工作。所以我通常有Jupyter笔记本

54:05 and VS code, I develop a Jupyter notebook, I paste into VS code, and then I kick off
和VS代码,我开发了一本Jupyter笔记本, 我粘贴到VS代码中, 然后我就开球

54:09 experiments from the repo, of course, from the code repository. So that's roughly some
当然,从代码存储处的回购实验中, 也有一些。

54:15 notes on the development process of working with neural nets. Lastly, I think this lecture
最后,我认为这次演讲

54:19 unlocks a lot of potential for their lectures, because number one, we have to convert our neural
释放了他们演讲的巨大潜力 因为第一,我们必须改变神经

54:24 network to actually use these dilated causal convolutional layers. So implementing the COM
网络, 以实际使用这些膨胀的因果变化层。 所以执行 COM

54:29 net. Number two, potentially starting to get into what this means, where are residual
净二号,可能开始进入这意味着什么的阶段,

54:34 connections, it's good connections and why are they useful. Number three, we, as I mentioned,
连接良好,为什么它们有用?

54:40 we don't have any experimental harness. So right now I'm just guessing checking everything.
我们没有实验性工具,所以现在我猜所有东西都检查了

54:44 This is not representative of typical deep learning workflows. You have to set up your
这不是典型的深层学习工作流程的代表。 您必须设置您的

54:48 evaluation harness, you can kick off experiments, you have lots of arguments that your script can
评估利用,你可以启动实验, 你有很多论点,你的脚本可以

54:52 take, you're kicking off a lot of experimentation, you're looking at a lot of plots of training
开始很多实验,你正在研究许多训练

**training** ['treinin]
训练 确定构成模型的理想参数 的过程。
神经网络基础

54:57 and validation losses, and you're looking at what is working and what is not working.
和验证损失, 你正在看什么是有效的,什么是不起作用的。

**validation**
验证 验证
训练与优化

55:00 And you're working on this like population level. And you're doing all these hyper
你正在做这个,就像人口水平。

55:04 parameter searches. And so we've done none of that so far. So how to set that up and how to make it
参数搜索。 所以我们至今没有做任何那些, 所以如何设置和如何建立

55:10 good, I think it's a whole another topic. And number three, we should probably cover recurrent
说第三,我们也许应该覆盖

55:15 neural networks, our names, LSTM's, Grooves and of course, Transformers. So many places to go.
神经网络,我们的名字,LSTM的,Grooves,当然还有变形器。

## 隐藏层 · 神经网络 55:23

55:23 And we'll cover that in the future. For now, sorry, I forgot to say that if you are
以后我们会再讨论这个的,抱歉,我忘了说 如果你是

55:28 interested, I think it is kind of interesting to try to beat this number 1.993, because I
有兴趣,我觉得尝试打这个号码1.993 很有趣 因为我

55:33 really haven't tried a lot of experimentation here and there's quite a bit of longing for it
实在没试过很多实验,还挺渴望的

55:37 potentially to still persist further. So I haven't tried any other ways of allocating these channels
可能还会继续。所以我没有尝试过其他方法来分配这些频道

55:43 in this neural net, maybe the number of dimensions for the embedding is all wrong.
可能嵌入的维度数完全错了

**neural net**
神经网络 神经网络
神经网络基础

**embedding** [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

55:48 Maybe it's possible to actually take the original network with just one hidden layer
也许可以把原始网络 隐藏一个层

**hidden layer**
隐藏层 神经网络 中的合成层，介于输入层 （即特征）和输出层 （即预测）之间。神经网络包含一个或多个隐藏层。
神经网络基础

55:52 and make it big enough and actually beat my fancy hierarchical network. It's not obvious.
让它足够大,并且实际上打败了我的高档网络。

55:57 That would be kind of embarrassing if this did not do better even once you torture it a
那样就算你折磨它,也好不到哪去

56:01 little bit. Maybe you can read the WaveNet paper and try to figure out how some of these layers work
略微一点点。 也许你可以读一下WaveNet的纸, 并试着找出这些层是如何工作的

56:05 and implement them yourselves using all we have. And of course, you can always tune some of the
当然,你总是可以调制一些

56:10 initialization or some of the optimization and see if you can improve it that way. So I'd be
初始化或优化的某个部分, 并看看你能否改进它。 所以我会

56:16 curious if people can come up with some ways to beat this. And yeah, that's it for now. Bye.
好奇人们能不能想出一些办法来克服这个。是的,现在就这样了。再见。

点击任意句子从该处播放 · 空格 播放/暂停 · T 中文 · C 连播 · ↑ ↓ 上下句 · Esc 停止
字幕由 ASR 转录并经 AI 校正重译，术语释义仅供参考。